

CarPlay 未経験から始める車載 UI 入門

車載アプリ開発の制約と面白さ

西凜平 / Kohei Nishi / @hiiragi589

CarPlay は、iPhone アプリをそのまま車載ディスプレイに映す仕組みではありません。「運転中に利用したい機能」という目的に対して、車内でユーザーに示す情報を設計する開発領域です。この記事では、番組一覧からエピソードを選んで再生し、車側から再生や一時停止を操作できる、ポッドキャスト風の音声アプリを題材にします。このサンプルアプリを通して、一覧の表示、画面遷移、Now Playing、システムとの再生状態の共有という CarPlay 開発の基本を、小さな構成でひと続きに確認できます。

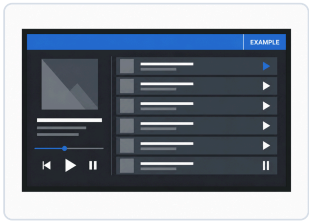
CarPlay はアプリのミラーリングではない

CarPlay は、iPhone の画面をそのまま車に映す仕組みではありません。「運転中に利用したい機能」という目的を先に決め、その目的に合わせて「ユーザーに示す情報」を設計します。アプリ側は CarPlay Framework の提供する Template やメディア情報を使い、この設計を車載ディスプレイ向けの体験として別に組み立てます。

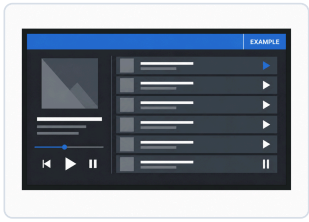
通常の iOS アプリなら、画面を自由に作り、ラベルやボタンを好きなところに配置できます。しかし車内でユーザーが集中すべきなのは、アプリの機能を使うことではなく運転することです。CarPlay では、便利そうな機能を足すよりも、運転中または停車中の文脈で本当に必要な情報だけを残すことが出発点になります。



Template で作る最小フロー



説明用イメージ



説明用イメージ

上の図は画面遷移を説明するための抽象的なイメージであり、CarPlay の実際の UI スクリーンショットではありません。

最初に触る CarPlay UI として分かりやすいのは、一覧から詳細へ進む流れです。ここでは、ポッドキャスト風の音声アプリを題材に、2つの `CPListTemplate` と `CPNowPlayingTemplate` を1本の体験としてつなげます。

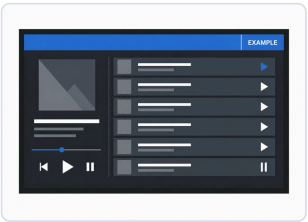
サンプル画面	使う API	題材での用途
エピソード一覧（ルート）	<code>CPListTemplate</code>	エピソードの一覧を表示する
詳細	<code>CPListTemplate</code>	長さや更新日を見せ、再生を起動する
再生中	<code>CPNowPlayingTemplate</code>	運転中のメディア操作を受ける
遷移	<code>CPInterfaceController</code>	画面の push / pop を管理する

Root Template を置く

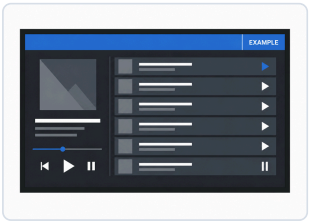
前ページで接続時に起動した Coordinator は、`makeHomeTemplate()` で Root Template を作ります。表示したいデータを `CPListItem` に変換し、セクションにまとめて `CPListTemplate` に渡します。

```
private func makeHomeTemplate() -> CPListTemplate {
    let episodes = Episode.samples.prefix(
        CPListTemplate.maximumItemCount
    )
    let items = episodes.map { episode in
        let item = CPListItem(
            text: episode.title,
            detailText: episode.subtitle
        )
        item.handler = { [weak self] _, completion
            in
                self?.showDetail(of: episode)
                completion()
            }
        return item
    }
    return CPListTemplate(
        title: "Audio Sample",
        sections: [CPListSection(items: items)]
    )
}
```

項目がタップされたことは `handler` クローージャに伝えられます。`handler` では「次の画面へ進む」処理を実行し、最後に `completion` を呼んでタップ処理の完了を OS に伝えます。`handler` はメインキューで実行されるため、ネットワーク取



説明用イメージ



説明用イメージ

得のような時間のかかる処理は別のキューへ移し、処理が終わってから `completion` を呼びます。

一覧は選ぶ場所

iPhone 側なら、ランキング、検索、履歴、バナー、設定を置きたくなるかもしれません。CarPlay 側では、最初の判断を少なくします。一覧は「詳しく読む場所」ではなく、表示された選択肢から次に行う操作を選ぶ場所と考えます。

表示件数にも上限があります。上限はシステム側が決めるため、`CPListTemplate.maximumItemCount` で確認し、超える分は渡す前に絞ります。「全部見せる」から「今選べる分だけ見せる」への切り替えが、CarPlay の一覧設計です。

詳細は行動の直前に絞る

候補を選んだら、もう1つの `CPListTemplate` を push します。詳細画面といっても、長い説明や複雑な編集は置きません。「このエピソードを再生してよいか」を判断できるだけの情報と、次のアクションを残します。

```
private func showDetail(of episode: Episode) {
    let playItem = CPListItem(
        text: "再生",
        detailText: "Now Playing を開く"
    )
    playItem.handler = { [weak self] _, completion
        in
            self?.startPlayback(of: episode)
            completion()
        }
    let detail = CPListTemplate(
        title: episode.title,
        sections: [
            CPListSection(
                items: metadataItems,
                header: "エピソード情報",
                sectionIndexTitle: nil
            ),
            CPListSection(
                items: [playItem],
                header: "アクション",
                sectionIndexTitle: nil
            )
        ]
    )
    interfaceController.pushTemplate(
        detail, animated: true, completion: nil
    )
}
```

`metadataItems` には、長さ、更新日、作者を表示する `CPListItem` を渡します。Audio カテゴリで利用できる Template に合わせながら、情報とアクションをセクションで

Now Playing で再生体験をつなぐ

一覧でエピソードを選び、再生を始め、車側の操作と再生状態を同期するところまで進みます。ここで Template の表示と音声再生の仕組みをつなぐのが `CPNowPlayingTemplate` です。

`CPListTemplate` は必要に応じて生成しますが、`CPNowPlayingTemplate` は自分で生成せず、Apple が提供する共有インスタンス `shared` を使います。生成コードに別の初期化方法が現れた場合は、公式ドキュメントに示された使用方法を優先します。

```
private func startPlayback(of episode: Episode) {
    player.play(episode)
    interfaceController.pushTemplate(
        CPNowPlayingTemplate.shared,
        animated: true, completion: nil
    )
}
```

再生状態はシステムに渡す

`CPNowPlayingTemplate` は再生 UI の表示先です。画面に出るエピソード名、再生位置、再生状態は自動では入らないため、`MPNowPlayingInfoCenter` 経由でシステムに伝えます。

```
func refresh() {
    guard let episode =
        player.currentEpisode
    else { return }
    let center = MPNowPlayingInfoCenter.default()
    center.nowPlayingInfo = [
        MPMediaItemPropertyTitle: episode.title,
        MPMediaItemPropertyArtist: episode.author,
        MPMediaItemPropertyPlaybackDuration:
            episode.duration,

        MPNowPlayingInfoPropertyElapsedPlaybackTime:
            player.currentTime,
        MPNowPlayingInfoPropertyPlaybackRate:
            player.isPlaying ? Double(player.rate)
        : 0,

        MPNowPlayingInfoPropertyDefaultPlaybackRate:
            Double(player.rate),
    ]
}
```

`PlaybackRate` は現在の状態を、`DefaultPlaybackRate` は選択中の通常再生速度を表します。サンプルでは再生開始、一時停止、再開、速度変更のたびに `refresh()` を呼び、システムへ最新値を渡します。

車側やシステム UI からの操作は、`MPRemoteCommandCenter` にコマンドとして届きます。再生は `playCommand`、一時停止

は `pauseCommand` で受け取り、その中でプレイヤーを動かします。ロック画面や AirPods の操作と同じ、普段の iOS 開発でおなじみの仕組みです。

```
let center = MPRemoteCommandCenter.shared()
center.playCommand.addTarget { [weak self] _ in
    self?.player.resume()
    return .success
}
center.pauseCommand.addTarget { [weak self] _ in
    self?.player.pause()
    return .success
}
```

再生速度の変更など、対応する操作の分だけ同じ形でコマンドを登録します。Remote Command を含むコード全体は、記事末尾の QR コードから GitHub リポジトリを開いて確認できます。

ボタンは絞る

CarPlay 側に出すボタンも、置けるから置くものではありません。再生速度、次に再生、ライブラリ追加など、車内で本当に使う操作に絞ります。

```
CPNowPlayingTemplate.shared.updateNowPlayingButtons {
    [
        CPNowPlayingPlaybackRateButton { [weak self] _
            in
                self?.player.cycleRate()
        },
    ]
}
```

判断基準はここでも同じです。iPhone 側で豊かに管理し、CarPlay 側では短い操作にする。

まとめ

何を表示しないかを先に決め、Scene で接続を受け、必要な Template だけを組み合わせる。制約を前提に設計すれば、CarPlay は今日から学び始められます。

動くサンプルコードを GitHub で公開

紙面で扱った一覧、詳細、Now Playing のフローを実装したサンプルアプリを GitHub で公開しています。README では、動作環境、Entitlement、Simulator 手順、ハマりどころ、公式リンクも確認できます。



サンプルコード・README

`CPListTemplate` から Now Playing まで、実際に動かして確認できます。再生される機械的な音声は、再生機能の確認用に生成したダミー音源です。



筆者について

hiiragi589

2001 年、和歌山生まれ。高校で C 言語に出会う。大学・大学院では競技プログラミング、XR 制作、ロボットアーム制御を経験。仕事では、バックエンドと Android を経て iOS 開発へ。